Student Information

Student ID 25000148967

GitHub Repository Link https://github.com/robwhite-lis/AI_Futures_Assessment_1

Predicting Melanoma Patient Survival Outcomes Using Logistic Regression: A Classification Analysis of the Veneto Tumour Registry

Introduction

Motivation

The purpose of this notebook is to see if Logistic Regression can be used to predict survival of a patient (at December 31st 2020) from sociodemographic (age and sex) and clinical pathological features (histology, site etc.).

The Dataset

The dataset used is from the Veneto Tumour Registry created in 2010. Further details are provided in Data_Description.md in this repository.

The registry is of neoplastic disease, i.e. diseases involving the uncontrolled growth of cells, which when malignant lead to cancer.

Study population: all incident cases of invasive cutaneous malignant melanoma (CMM) recorded in the Veneto Tumour Registry (Registro Tumori del Veneto – RTV) in 2015 (1,050 cases) and 2017 (1,205 cases) for which the number of mitoses was available.

In [1]:
```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import io
import requests
import matplotlib.pyplot as plt

import seaborn as sns
sns.set()

df_melanoma = pd.read_csv('melanoma_mitoses.csv', sep=';')
```

In [2]:
```
# check the dataframe has loaded correctly
df_melanoma.head()
```

Out[2]:

	ID_prog	age	sex	year	histology	site	pT	pN	pM	TNM_stage	breslow	cl
0	1	31	F	2015	Nodular melanoma	Trunk	pT4	N3c	M0	III	>=4	
1	2	53	M	2017	Superficial spreading melanoma	Trunk	pT1	N0	M0	I	< 0.76	
2	3	67	F	2015	Superficial spreading melanoma	Upper limb	pT1	N0	M0	I	0.76-1.50	
3	4	29	F	2015	Superficial spreading melanoma	Trunk	pT1	N1a	M0	III	< 0.76	
4	5	57	F	2015	Superficial spreading melanoma	Lower limb	pT1	N0	M0	I	< 0.76	

In [3]:
```
# check the shape of the dataframe
df_melanoma.shape
```

Out[3]:
```
(2255, 20)
```

In [4]:
```
# and the columns...
df_melanoma.columns
```

Out[4]:
```
Index(['ID_prog', 'age', 'sex', 'year', 'histology', 'site', 'pT', 'pN', 'pM',
       'TNM_stage', 'breslow', 'clark', 'ulcerazione', 'regressione', 'til',
       'growth', 'positive_slnb', 'mitoses', 'status', 'survival'],
      dtype='object')
```

Encoding the data

Logistic Regression works on numeric data, the notebook will use the optimum encoding for the different data. Redundant columns will be dropped. The justification for the selection of those columns is in the table below.

Column	Explanation
survival	Leakage: another way of showing status the variable we are trying to predict. We choose 'status' instead as it is a classification not a continous variable.
ID_prog	Row identifier, no predictive value.
year	Although this is year of diagnosis, which is another way of showing survival (i.e. leakage) and furthermore it is not a clinical signal.

In [5]:
```
# First drop unnecessary columns
df_melanoma_clean = df_melanoma.drop(columns=[ 'survival',    # Leakage: another
                                               'ID_prog',     # Row identifier, n
                                               'year'])       # Although this is
```

```
# ORDINAL ENCODINGS
# This for the variables where we care about the order
# so e.g. the larger the breslow_map (thicker the tumor)
# the more dangerous it is
tnm_map = {'I': 1, 'II': 2, 'III': 3, 'IV': 4}
df_melanoma_clean['TNM_stage'] = df_melanoma_clean['TNM_stage'].map(tnm_map)

pt_map = {'pT1': 1, 'pT2': 2, 'pT3': 3, 'pT4': 4}
df_melanoma_clean['pT'] = df_melanoma_clean['pT'].map(pt_map)

breslow_map = {'< 0.76': 1, '0.76-1.50': 2, '1.51-3.99': 3, '>=4': 4}
df_melanoma_clean['breslow'] = df_melanoma_clean['breslow'].map(breslow_map)

clark_map = {'I': 1, 'II': 2, 'III': 3, 'IV': 4, 'V': 5}
df_melanoma_clean['clark'] = df_melanoma_clean['clark'].map(clark_map)

pn_map = {'N0': 0, 'N1a': 1, 'N1b': 1, 'N1c': 1,
          'N2a': 2, 'N2b': 2, 'N2c': 2,
          'N3':  3, 'N3c': 3}
df_melanoma_clean['pN'] = df_melanoma_clean['pN'].map(pn_map)


# BINARY ENCODINGS
# Simple categorization between 2 options
df_melanoma_clean['sex']         = df_melanoma_clean['sex'].map({'F': 0, 'M': 1}
df_melanoma_clean['ulcerazione'] = df_melanoma_clean['ulcerazione'].map({'Absent
df_melanoma_clean['til']         = df_melanoma_clean['til'].map({'Absent': 0, 'P
df_melanoma_clean['growth']      = df_melanoma_clean['growth'].map({'Radial': 0,
df_melanoma_clean['regressione'] = df_melanoma_clean['regressione'].map({'Absent
df_melanoma_clean['pM']          = df_melanoma_clean['pM'].map({'M0': 0, 'M1': 1
df_melanoma_clean['positive_slnb'] = df_melanoma_clean['positive_slnb'].map({'0'


# ONE-HOT ENCODE
# Columns we need to encode but don't care about the order
# This is a standard method of working with this kind of data
# drop_first=True drops one dummy per group to avoid multicollinearity
# (i.e. you can determine this columns value from the state of the others)
df_melanoma_clean = pd.get_dummies(df_melanoma_clean, columns=['histology', 'sit

# LOG-TRANSFORM
# This is the recommended way where you have a wide range of values
# mitosis runs from 0 to 55 and is very skewed to the lower end of
# that range
df_melanoma_clean['mitoses_log'] = np.log1p(df_melanoma_clean['mitoses'])

# ... and drop the original mitosis column
df_melanoma_clean = df_melanoma_clean.drop(columns=['mitoses'])
```

In [6]:
```
df_melanoma_clean.columns
```

Out[6]:
```
Index(['age', 'sex', 'pT', 'pN', 'pM', 'TNM_stage', 'breslow', 'clark',
       'ulcerazione', 'regressione', 'til', 'growth', 'positive_slnb',
       'status', 'histology_Desmoplastic melanoma',
       'histology_Lentigo maligna', 'histology_Malignant melanoma',
       'histology_Nodular melanoma', 'histology_Spitzoid melanoma',
       'histology_Superficial spreading melanoma', 'site_Head',
       'site_Lower limb', 'site_Trunk', 'site_Upper limb', 'mitoses_log'],
      dtype='object')
```

In [7]:
```
df_melanoma_clean.shape
```

Out[7]:
```
(2255, 25)
```

Data Problems fitting the model.

Logistic regression does not accept NULLS or NaN values, causing errors when we try to "fit" the model. First of all identify the columns causing the issue.

In [8]:
```
df_melanoma_clean.isnull().sum()
```

Out[8]:
```
age                                          0
sex                                          0
pT                                           3
pN                                           8
pM                                          13
TNM_stage                                   16
breslow                                      4
clark                                      206
ulcerazione                                 25
regressione                                555
til                                        143
growth                                     390
positive_slnb                                0
status                                       0
histology_Desmoplastic melanoma              0
histology_Lentigo maligna                    0
histology_Malignant melanoma                 0
histology_Nodular melanoma                   0
histology_Spitzoid melanoma                  0
histology_Superficial spreading melanoma     0
site_Head                                    0
site_Lower limb                              0
site_Trunk                                   0
site_Upper limb                              0
mitoses_log                                  0
dtype: int64
```

Missing values analysis

Having identified the missing values, there are a number of standard methods of dealing with nulls. First some context...

Data Science there are different classifications of missing data..

Classification	Definition
Missing Completely at Random (MCAR)	The distribution of missing values has no relationship to anything
Missing at Random (MAR)	The missing values depends on other observed variables
Missing Not at Random (MNAR)	the nature of the missing variables is unknown

Thinking in these terms will allow us to make some assumptions about the data. E.g. Clark level might be missing more often for thicker tumours because clinicians didn't bother recording it when Breslow was clearly the more relevant measure. This is an example of MAR, so it is reasonable to remove the feature.

Beyond that however we can not draw conclusions about the other features given a clinician may have not thought them important enough to record, or may have mistakenly not recorded anything instead of positively affirming the absence of a characteristic.

One method to compensate for missing date is imputation. "...imputation is the process of replacing missing data with substituted values" https://en.wikipedia.org/wiki/Imputation_(statistics) and is often used to fill in missing data. Simple imputation is rule based e.g. taking the mean average of the variable, whilst Multiple Imputation relies on generating multiple copies of the data. Neither of these methods are appropriate in our case as we don't have a clear expectation that a missing value is related to other data recorded elsewhere on the data row for the patient. Hence we cannot establish the case for missing data being MAR, and imputation is not suitable (https://pmc.ncbi.nlm.nih.gov/articles/PMC6293424/)

The other option open to us is simply "remove" the missing data, i.e. drop features columns with high frequency of nulls and drop the rows containing nulls.

Which data to remove

Looking at the data above we can see the 3 main offenders are regressione (regression), growth and clark. Clark is an easy one discount as it is an alternate and less accurate measure of Breslow (both refer to the thickness of the tumour). Regression and growth are less easy to discount but the number of rows affected makes deleting the rows impractical as it would severely reduce the size of the dataset. Therefore we will drop these columns.

In [9]:
```
# Based on that let's clean up some more
df_melanoma_clean = df_melanoma_clean.drop(columns=[ 'clark',      # this is an
                                                     'regressione', # there are
                                                     'growth'])      # same argu
```

In [10]:
```
df_melanoma_clean.columns
```

Out[10]:
```
Index(['age', 'sex', 'pT', 'pN', 'pM', 'TNM_stage', 'breslow', 'ulcerazione',
       'til', 'positive_slnb', 'status', 'histology_Desmoplastic melanoma',
       'histology_Lentigo maligna', 'histology_Malignant melanoma',
       'histology_Nodular melanoma', 'histology_Spitzoid melanoma',
       'histology_Superficial spreading melanoma', 'site_Head',
       'site_Lower limb', 'site_Trunk', 'site_Upper limb', 'mitoses_log'],
      dtype='object')
```

Now we can rerun the check to see how many rows we'd lose by dropping all rows that contain nulls.

In [11]:
```
df_melanoma_clean.isnull().sum()
```

Out[11]:
```
age                                          0
sex                                          0
pT                                           3
pN                                           8
pM                                          13
TNM_stage                                   16
breslow                                      4
ulcerazione                                 25
til                                        143
positive_slnb                                0
status                                       0
histology_Desmoplastic melanoma              0
histology_Lentigo maligna                    0
histology_Malignant melanoma                 0
histology_Nodular melanoma                   0
histology_Spitzoid melanoma                  0
histology_Superficial spreading melanoma     0
site_Head                                    0
site_Lower limb                              0
site_Trunk                                   0
site_Upper limb                              0
mitoses_log                                  0
dtype: int64
```

In [12]:
```
# Check the size of the dataframe
df_melanoma_clean.shape
```

Out[12]:
```
(2255, 22)
```

In [13]:
```
# drop rows that contain a null value and recheck size
df_melanoma_clean = df_melanoma_clean.dropna()
df_melanoma_clean.shape
```

Out[13]:
```
(2079, 22)
```

In [14]:
```
df_melanoma_clean['status'].value_counts()
```

Out[14]:
```
status
0    1821
1     258
Name: count, dtype: int64
```

In [15]:
```
# check we have removed nulls
df_melanoma_clean.isnull().sum()
```

Out[15]:
```
age                                        0
sex                                        0
pT                                         0
pN                                         0
pM                                         0
TNM_stage                                  0
breslow                                    0
ulcerazione                                0
til                                        0
positive_slnb                              0
status                                     0
histology_Desmoplastic melanoma            0
histology_Lentigo maligna                  0
histology_Malignant melanoma               0
histology_Nodular melanoma                 0
histology_Spitzoid melanoma                0
histology_Superficial spreading melanoma   0
site_Head                                  0
site_Lower limb                            0
site_Trunk                                 0
site_Upper limb                            0
mitoses_log                                0
dtype: int64
```

👍 No NULL values

Next Train the Model

In [16]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report


# Prepare data status is the column we are testing for

X = df_melanoma_clean.drop(columns=['status'])

# y is a series (aka a list) for target variable.
y = df_melanoma_clean['status']

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

Training and Test Set Characteristics

Looking at the relative sizes of the data sets and composition will highlight insights for our analysis. Recall Status=0 is survived, status=1 is deceased

In [17]:
```
print(f"Training set size: {len(X_train)}")
print(f"Test set size: {len(X_test)}")

print(f"Training set status values: {y_train.value_counts()}")
print(f"Test set status values: {y_test.value_counts()}")
```

```
Training set size: 1663
Test set size: 416
Training set status values: status
0    1462
1     201
Name: count, dtype: int64
Test set status values: status
0    359
1     57
Name: count, dtype: int64
```

In [18]:
```
# Fit logistic regression model
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)
```

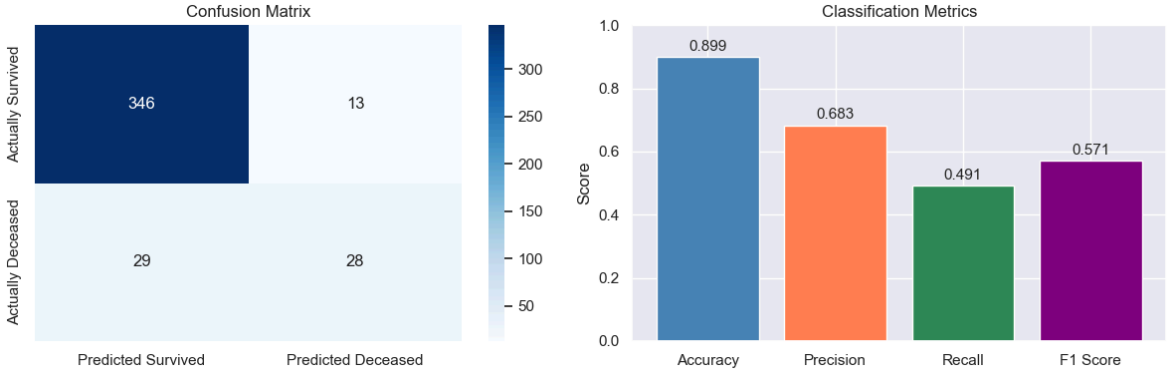
Out[18]:
```
▾ LogisticRegression
▶ Parameters
```

In [19]:
```
# Make predictions on test set
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]  # Probability of class 1
```

Visualize the output

In [20]:
```
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_

# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Left: Confusion matrix heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
           xticklabels=['Predicted Survived', 'Predicted Deceased'],
           yticklabels=['Actually Survived', 'Actually Deceased'])
axes[0].set_title('Confusion Matrix')

# Right: Metrics visualization
metrics = {
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision': precision_score(y_test, y_pred),
    'Recall': recall_score(y_test, y_pred),
    'F1 Score': f1_score(y_test, y_pred)
}

bars = axes[1].bar(metrics.keys(), metrics.values(), color=['steelblue', 'coral'
axes[1].set_ylim(0, 1)
axes[1].set_ylabel('Score')
axes[1].set_title('Classification Metrics')
for bar, val in zip(bars, metrics.values()):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
                 f'{val:.3f}', ha='center', fontsize=11)

plt.tight_layout()
plt.show()
```

```
# Extract values from confusion matrix
TN, FP, FN, TP = cm.ravel()

print(f"  True Negatives (TN): {TN}  — predicted Survived and actually Survived"
print(f"  False Positives (FP): {FP}  — predicted Deceased but actually Survived
print(f"  False Negatives (FN): {FN}  — predicted Survived but were actually Dec
print(f"  True Positives (TP):  {TP}  — predicted Deceased and actually Deceased
```




```
  True Negatives (TN): 346  — predicted Survived and actually Survived
  False Positives (FP): 13  — predicted Deceased but actually Survived
  False Negatives (FN): 29  — predicted Survived but were actually Deceased (miss
ed deaths)
  True Positives (TP):  28  — predicted Deceased and actually Deceased
```

In [21]:

```
# Evaluate performance
print("Classification Report on Test Set:")
print("=" * 50)
print(classification_report(y_test, y_pred, target_names=['Survived', 'Deceased'
```

```
Classification Report on Test Set:
==================================================
              precision    recall  f1-score   support

    Survived       0.92      0.96      0.94       359
    Deceased       0.68      0.49      0.57        57

    accuracy                           0.90       416
   macro avg       0.80      0.73      0.76       416
weighted avg       0.89      0.90      0.89       416
```

Results

Definitions of the Classification Report

Classification	Definition	Comments
Precision	For everything the model *predicted* as that class, how many were actually correct	"Deceased": of all the patients the model flagged as deaths, 68% genuinely died. The other 32% were false alarms (survivors incorrectly labelled as deaths)
Recall	Everything that *actually is* that class, how many did the model catch?	Of the 57 real deaths in the test set, the model only found 49% of them. Missing more than half from a clinical perspective is dangerous those are patients who died but the model said they'd survive, thus misleading the clinician.

Classification	Definition	Comments
F1-score	Harmonic mean of precision and recall. It penalises you if either one is poor.	The "Deceased" F1 of 0.57 suggests the model is struggling with the "Deceased" grouping, as this is much lower than the score for "Survived"
Support	Actual count of that class in your test set.	In the test dataset there are 359 survivors and 57 deaths
Accuracy	Overall, what fraction of all 416 predictions were correct.	90% is flattering the model a model that predicted "Survived" for every single patient would score 359/416 = 86% accuracy. This demonstrates a weakness for "Accuracy" in this kind of data
Macro average	Averages precision, recall, and F1 across both classes equally (i.e. is unweighted average)	Statistically, this treats "Survived" and "Deceased" as equally important and the macro recall of 0.73 is nowhere near as good as 90% accuracy suggests

| **Weighted average** | Averages the same metrics but weights each class by its support count | Because "Survived" (359 cases) dominates, the weighted average is pulled strongly toward those numbers and flatters the model. This is why it sits close to the accuracy figure.

Initial Conclusions

The model is behaving poorly due to the imbalance in the data between the survivors (1821) and deceased (258) in the original "clean" dataframe. The standard way of fixing this is to add the class_weight paramter to the LogisticRegression model, so that is what we will try next...

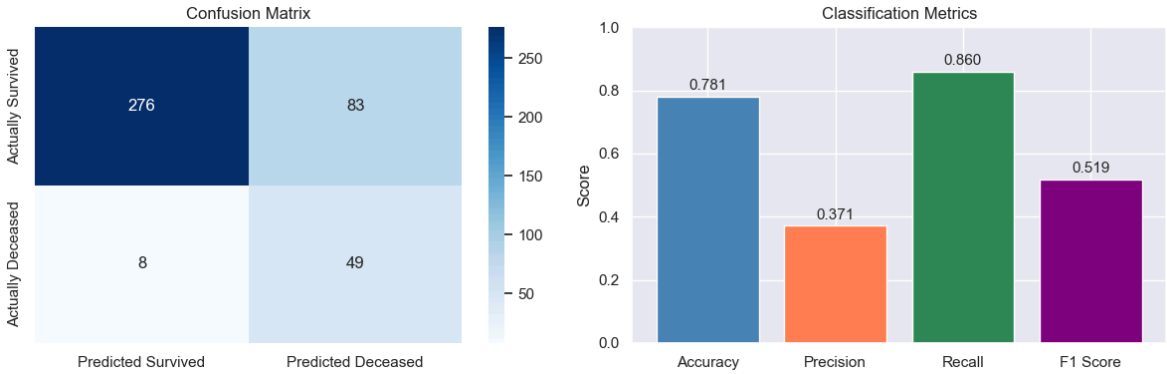
In [22]:
```
# Fit logistic regression model
model = LogisticRegression(class_weight='balanced', random_state=42)
model.fit(X_train, y_train)
```

Out[22]:
```
▾ LogisticRegression
▶ Parameters
```

In [23]:
```
# Make predictions on test set
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]  # Probability of class 1
```

In [24]:
```
# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Left: Confusion matrix heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Predicted Survived', 'Predicted Deceased'],
```

```
            yticklabels=['Actually Survived', 'Actually Deceased'])
axes[0].set_title('Confusion Matrix')

# Right: Metrics visualization
metrics = {
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision': precision_score(y_test, y_pred),
    'Recall': recall_score(y_test, y_pred),
    'F1 Score': f1_score(y_test, y_pred)
}

bars = axes[1].bar(metrics.keys(), metrics.values(), color=['steelblue', 'coral'
axes[1].set_ylim(0, 1)
axes[1].set_ylabel('Score')
axes[1].set_title('Classification Metrics')
for bar, val in zip(bars, metrics.values()):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
                 f'{val:.3f}', ha='center', fontsize=11)

plt.tight_layout()
plt.show()

# Extract values from confusion matrix
TN, FP, FN, TP = cm.ravel()

print(f"  True Negatives (TN): {TN}  — predicted Survived and actually Survived"
print(f"  False Positives (FP): {FP}  — predicted Deceased but actually Survived
print(f"  False Negatives (FN): {FN}  — predicted Survived but were actually Dec
print(f"  True Positives (TP):  {TP}  — predicted Deceased and actually Deceased
```




```
  True Negatives (TN): 276  — predicted Survived and actually Survived
  False Positives (FP): 83  — predicted Deceased but actually Survived
  False Negatives (FN): 8  — predicted Survived but were actually Deceased (misse
d deaths)
  True Positives (TP):  49  — predicted Deceased and actually Deceased
```

In [25]:

```
# Evaluate performance
print("Classification Report on Test Set:")
print("=" * 50)
print(classification_report(y_test, y_pred, target_names=['Survived', 'Deceased'
```

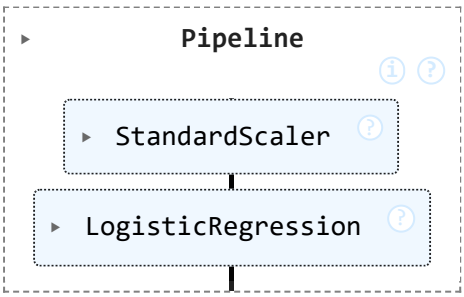
```
Classification Report on Test Set:
==================================================
              precision    recall  f1-score   support

    Survived       0.97      0.77      0.86       359
    Deceased       0.37      0.86      0.52        57

    accuracy                           0.78       416
   macro avg       0.67      0.81      0.69       416
weighted avg       0.89      0.78      0.81       416
```

Results

This is better but still not there, we can do better. Logistic Regression works by computing a weighted sum of the learned coefficient and the feature (aka column) value. Hence when the range of the feature values varies, features which have higher numerical values can be unduly influential in the model, affecting the accuracy of the optimisation algorithm. This is known as the feature scale problem https://scikit-learn.org/stable/auto_examples/preprocessing/plot_scaling_importance.html

In [26]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

model = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(
        class_weight='balanced',
        solver='lbfgs',
        max_iter=1000,
        random_state=42
    ))
])

model.fit(X_train, y_train)
```

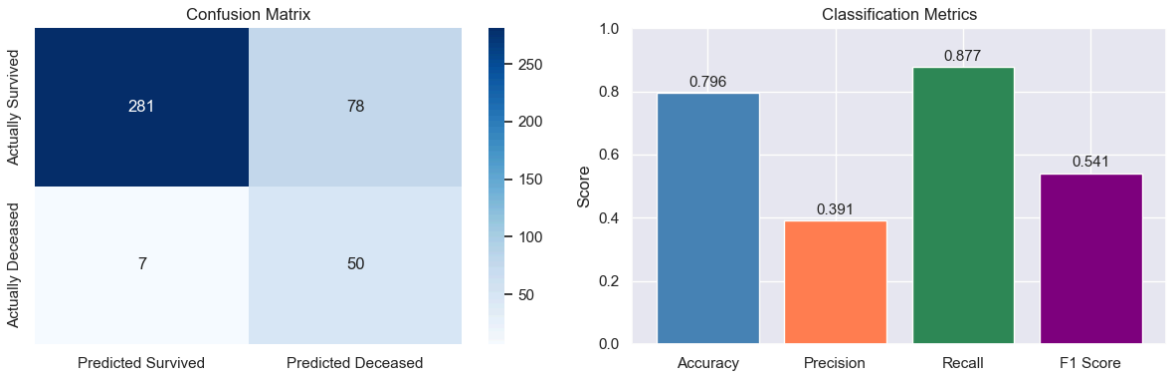
Out[26]:



In [27]:
```
# Make predictions on test set
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]  # Probability of class 1
```

In [28]:
```
# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Left: Confusion matrix heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
```

```
            xticklabels=['Predicted Survived', 'Predicted Deceased'],
            yticklabels=['Actually Survived', 'Actually Deceased'])
axes[0].set_title('Confusion Matrix')

# Right: Metrics visualization
metrics = {
    'Accuracy': accuracy_score(y_test, y_pred),
    'Precision': precision_score(y_test, y_pred),
    'Recall': recall_score(y_test, y_pred),
    'F1 Score': f1_score(y_test, y_pred)
}

bars = axes[1].bar(metrics.keys(), metrics.values(), color=['steelblue', 'coral'
axes[1].set_ylim(0, 1)
axes[1].set_ylabel('Score')
axes[1].set_title('Classification Metrics')
for bar, val in zip(bars, metrics.values()):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
                 f'{val:.3f}', ha='center', fontsize=11)

plt.tight_layout()
plt.show()

# Extract values from confusion matrix
TN, FP, FN, TP = cm.ravel()

print(f"  True Negatives (TN): {TN}  — predicted Survived and actually Survived"
print(f"  False Positives (FP): {FP}  — predicted Deceased but actually Survived
print(f"  False Negatives (FN): {FN}  — predicted Survived but were actually Dec
print(f"  True Positives (TP):  {TP}  — predicted Deceased and actually Deceased
```




```
  True Negatives (TN): 281  — predicted Survived and actually Survived
  False Positives (FP): 78  — predicted Deceased but actually Survived
  False Negatives (FN): 7  — predicted Survived but were actually Deceased (misse
d deaths)
  True Positives (TP):  50  — predicted Deceased and actually Deceased
```

In [29]:

```
# Evaluate performance
print("Classification Report on Test Set:")
print("=" * 50)
print(classification_report(y_test, y_pred, target_names=['Survived', 'Deceased'
```

```
Classification Report on Test Set:
==================================================
              precision    recall  f1-score   support

    Survived       0.98      0.78      0.87       359
    Deceased       0.39      0.88      0.54        57

    accuracy                           0.80       416
   macro avg       0.68      0.83      0.70       416
weighted avg       0.90      0.80      0.82       416
```

Conclusions

By tuning the parameters passed to the model the recall of the model has progressed from predicting 49% of deaths accurately to 88%. This has been done apparently at the expense of predicting Survived (96% to 78%) however the initial recall was inflated by the imbalance in the survived vs deceased data (showing that predicting survival with no model would give a recall of 86%).

Limitations

What this analysis has shown is why a classification model such as logistic regression, although useful for predicting survivability also has serious shortcomings for medical analysis of this nature. The binary nature of status, which was alive or deceased on 31st December 2020 treats a patient who has survived 1 year from diagnosis the same as one who has survived 5. Clearly from a patient perspective the latter is a much stronger outcome.

In reality for medical diagnosis we need more sophisticated models such as proportional hazards models https://en.wikipedia.org/wiki/Proportional_hazards_model where consideration of passing time before an event occurs is considered.